Universidad Galileo	Guatemala 13 de octubre 2026
Facultad: FISICC	Alumno: Luis Cordova
Curso: Microprocesadores	Carnet: 23012122
Sección: A	Hora de Laboratorio: 1:00 a 1:50
Auxiliar: Brandon Eduardo Mendoza Espinoza	Día de Laboratorio: Lunes

Laboratorio #8 Convertidor Analógico-Digital

Resumen:

Esta práctica giró alrededor del ADC del STM32G431. La idea era leer un potenciómetro y traducir esa lectura en cuatro LEDs que se encienden por tramos, repartiendo el recorrido en quintos. El programa completo está en ensamblador Thumb2, main incluido: no hay ni un archivo en C propio [4].

El enunciado incluye un anexo en C con la configuración del ADC y la lectura de una conversión. Ese anexo se pasó a ensamblador, pero al grabarlo en la tarjeta no arrancó bien, y hubo que arreglarle dos cosas: el canal que lee y el momento en que se configuran los registros. A eso se sumó un tercer problema, este sí propio de tener el main en ensamblador, que hacía que el programa ni siquiera llegara a correr.

Objetivos:

- Dejar el ADC del STM32G431 configurado a nivel de registro desde ensamblador: reloj, pin analógico, resolución, canal y secuencia, listo para convertir.
- Leer una conversión y traducir ese valor en el encendido de cuatro LEDs, repartiendo el recorrido del potenciómetro en quintos.
- Armar un programa entero en ensamblador, main incluido, y entender qué espera la librería de C para poder invocarlo.

Teoría:

Un ADC agarra un voltaje y lo devuelve como número. El STM32G431 tiene cinco convertidores de aproximaciones sucesivas, de hasta 12 bits, con varios canales multiplexados: un mismo convertidor puede medir varios pines, pero solo uno por vez, así que hay que decirle cuál [4].

La referencia de la tarjeta son 3.3 V. Eso fija los extremos: 0 V da el mínimo y 3.3 V el máximo. La cantidad de escalones intermedios la decide la resolución. A 12 bits son 4096 pasos; a 10 bits, 1024 [1].

Un potenciómetro no es más que una resistencia con un contacto que se desliza. Si sus extremos van a 3.3 V y a tierra, la pata del medio entrega un voltaje proporcional al giro, entre 0 y 3.3 V. Por eso es la entrada más cómoda para probar un ADC.

Poner el ADC a funcionar implica tocar varios registros. El reloj se habilita en RCC->AHB2ENR, igual que el de los puertos, porque en el STM32G4 el ADC cuelga del bus AHB2 [1]. El pin se deja en modo analógico poniendo los dos bits de GPIOx_MODER en 11 [1]. La resolución se elige en ADCn_CFGR; la cantidad de conversiones y el canal, en ADCn_SQR1; y la fuente de reloj del convertidor, en ADC12_COMMON->CCR [4].

El arranque del ADC no es instantáneo. Después del reset queda en un modo de bajo consumo llamado deep power down, con la circuitería apagada. Hay que sacarlo de ahí, encender su regulador interno y darle tiempo a estabilizarse; recién entonces se puede habilitar con ADEN en ADCn_CR y esperar a que avise con ADRDY en ADCn_ISR [1].

Con eso listo, cada conversión se dispara con ADSTART, se aguarda la bandera EOC y el resultado se recoge de ADCn_DR [4]. Leer ese registro limpia la bandera por hardware, así que en la siguiente vuelta del lazo no hay que borrarla a mano [1].

Resultados prácticos:

A continuación se presentan las capturas obtenidas durante el desarrollo y la ejecución de la práctica.

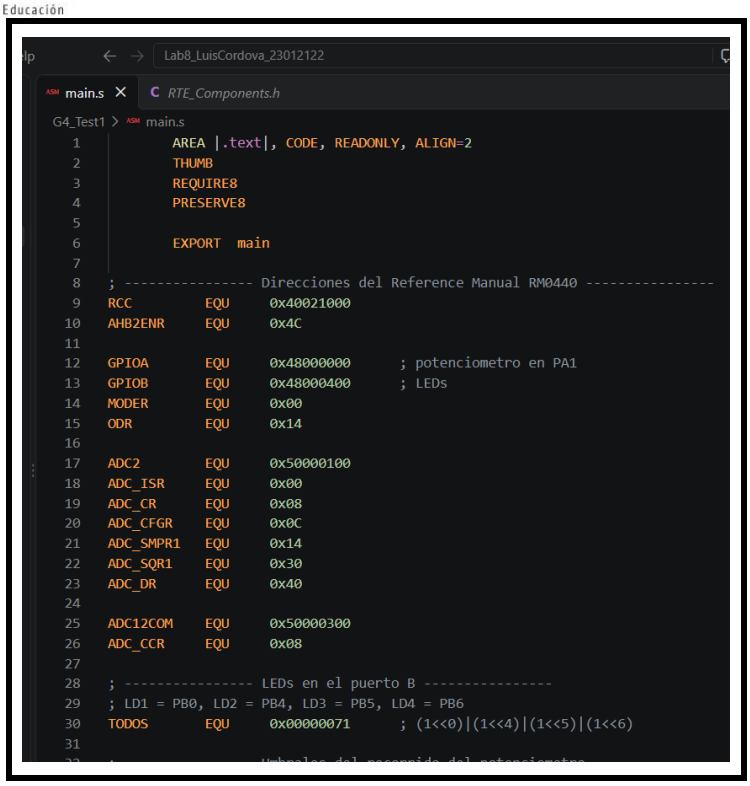


Ilustración 1- direcciones base y offsets definidos con EQU, tomados del manual de referencia.



Ilustración 2- umbrales de los quintos y lazo principal del programa.

```
; ================= Configuracion de los LEDs =================
leds_config
        push {lr}

        ;RCC->AHB2ENR |= (1<<1);   reloj del puerto B
        ldr  r0, =RCC                   ;read
        ldr  r1, [r0,#AHB2ENR]
        movs r2, #0x2                   ;modify
        orrs r1, r1, r2
        str  r1, [r0,#AHB2ENR]          ;write

        ;GPIOB->MODER: PB0, PB4, PB5 y PB6 como salida (01)
        ldr  r0, =GPIOB                 ;read
        ldr  r1, [r0,#MODER]
        ldr  r2, =0x00003F03            ;modify, los dos bits de cada pin
        bics r1, r1, r2
        ldr  r2, =0x00001501            ;el bit de abajo de cada par
        orrs r1, r1, r2
        str  r1, [r0,#MODER]            ;write

        ;GPIOB->ODR &= ~TODOS;   los cuatro LEDs apagados al empezar
        ldr  r1, [r0,#ODR]
        ldr  r2, =TODOS
        bics r1, r1, r2
        str  r1, [r0,#ODR]

        pop  {pc}
```

Ilustración 3- subrutina leds_config: reloj del puerto B y los cuatro pines como salida.

```
; ================= Configuracion del ADC =================
adc_config
        push {lr}

        ;RCC->AHB2ENR |= GPIOAEN | ADC12EN;   relojes del puerto A y del ADC
        ldr  r0, =RCC                   ;read
        ldr  r1, [r0,#AHB2ENR]
        movs r2, #0x1                   ;modify, GPIOAEN, bit 0
        orrs r1, r1, r2
        ldr  r2, =0x00002000            ;ADC12EN, bit 13
        orrs r1, r1, r2
        str  r1, [r0,#AHB2ENR]          ;write

        ;GPIOA->MODER |= (3<<2);   PA1 como entrada analogica (11)
        ldr  r0, =GPIOA                 ;read
        ldr  r1, [r0,#MODER]
        movs r2, #0x0C                  ;modify
        orrs r1, r1, r2
        str  r1, [r0,#MODER]            ;write

        ;ADC2->CR &= ~(1<<29);   salir del modo deep power down (antes de configurar)
        ldr  r0, =ADC2
        ldr  r1, [r0,#ADC_CR]
        ldr  r2, =0x20000000
        bics r1, r1, r2
        str  r1, [r0,#ADC_CR]
```

Ilustración 4- inicio de adc_config: relojes, PA1 como entrada analógica y salida del deep power down.

```
        ;ADC2->CR |= (1<<28);   encender el regulador de voltaje del ADC
        ldr  r1, [r0,#ADC_CR]
        ldr  r2, =0x10000000
        orrs r1, r1, r2
        str  r1, [r0,#ADC_CR]

        ;esperar a que el regulador se estabilice
        ldr  r3, =2000
espera_reg
        subs r3, r3, #1
        bne  espera_reg

        ;ADC12_COMMON->CCR |= (3<<16);   reloj del ADC = HCLK/4
        ldr  r2, =ADC12COM
        ldr  r1, [r2,#ADC_CCR]
        ldr  r3, =0x00030000
        orrs r1, r1, r3
        str  r1, [r2,#ADC_CCR]

        ;ADC2->CFGR |= (1<<12)|(1<<3);   OVRMOD y resolucion de 10 bits
        ldr  r1, [r0,#ADC_CFGR]
        ldr  r2, =0x00001008
        orrs r1, r1, r2
        str  r1, [r0,#ADC_CFGR]

        ;ADC2->SQR1 |= (2<<6);   una conversion (L=0) en el canal 2
        ldr  r1, [r0,#ADC_SQR1]
        movs r2, #0x80
        orrs r1, r1, r2
        str  r1, [r0,#ADC_SQR1]
```

Ilustración 5- regulador de voltaje, espera, reloj del ADC, resolución y canal.

```
        ;ADC2->SQR1 |= (2<<6);   una conversion (L=0) en el canal 2
        ldr  r1, [r0,#ADC_SQR1]
        movs r2, #0x80
        orrs r1, r1, r2
        str  r1, [r0,#ADC_SQR1]

        ;ADC2->SMPR1 |= (7<<6);   tiempo de muestreo largo para el canal 2
        ldr  r1, [r0,#ADC_SMPR1]
        ldr  r2, =0x000001C0
        orrs r1, r1, r2
        str  r1, [r0,#ADC_SMPR1]

        ;ADC2->CR |= (1<<0);   habilitar el ADC
        ldr  r1, [r0,#ADC_CR]
        movs r2, #0x1
        orrs r1, r1, r2
        str  r1, [r0,#ADC_CR]

        ;while(!(ADC2->ISR & ADRDY));   esperar a que el ADC este listo
espera_listo
        ldr  r1, [r0,#ADC_ISR]
        movs r2, #0x1
        ands r1, r1, r2
        beq  espera_listo

        pop  {pc}
```

Ilustración 6- tiempo de muestreo, habilitación del ADC y espera de la bandera ADRDY.

```
; ================ Una conversion, devuelve el dato en r0 ================
adc_read
        push {lr}

        ;ADC2->CR |= (1<<2);   iniciar la conversion
        ldr  r0, =ADC2                  ;read
        ldr  r1, [r0,#ADC_CR]
        movs r2, #0x4                   ;modify
        orrs r1, r1, r2
        str  r1, [r0,#ADC_CR]           ;write

        ;while(!(ADC2->ISR & EOC));   esperar a que termine
espera_fin
        ldr  r1, [r0,#ADC_ISR]
        movs r2, #0x4
        ands r1, r1, r2
        beq  espera_fin

        ;return ADC2->DR;
        ldr  r0, [r0,#ADC_DR]

        pop  {pc}
```

Ilustración 7- subrutina adc_read: inicia la conversión, espera EOC y devuelve el dato.

```
; ================ Enciende los LEDs segun el dato (r0) ================
leds_write
        push {lr}

        ldr  r1, =GPIOB                 ;read
        ldr  r2, [r1,#ODR]
        ldr  r3, =TODOS                 ;modify, apaga los cuatro
        bics r2, r2, r3

        ldr  r3, =UN_QUINTO             ; 0 a 1/5: todos apagados
        cmp  r0, r3
        blo  escribir
        movs r3, #0x40                  ; 1/5 a 2/5: LD4
        orrs r2, r2, r3

        ldr  r3, =DOS_QUINTOS
        cmp  r0, r3
        blo  escribir
        movs r3, #0x20                  ; 2/5 a 3/5: LD3 y LD4
        orrs r2, r2, r3

        ldr  r3, =TRES_QUINTOS
        cmp  r0, r3
        blo  escribir
        movs r3, #0x10
        orrs r2, r2, r3

        ldr  r3, =CUATRO_QUINTOS
        cmp  r0, r3
        blo  escribir
        movs r3, #0x01
        orrs r2, r2, r3
```

Ilustración 8- subrutina leds_write: compara el dato contra los cuatro umbrales.

```
escribir
        str  r2, [r1,#ODR]              ;write
        pop  {pc}


        AREA    |.ARM.use_no_argv|, DATA, READWRITE, ALIGN=2
        EXPORT  __ARM_use_no_argv
__ARM_use_no_argv
        DCD     1

        end
```

Ilustración 9- escritura al puerto y el símbolo __ARM_use_no_argv al final del archivo.

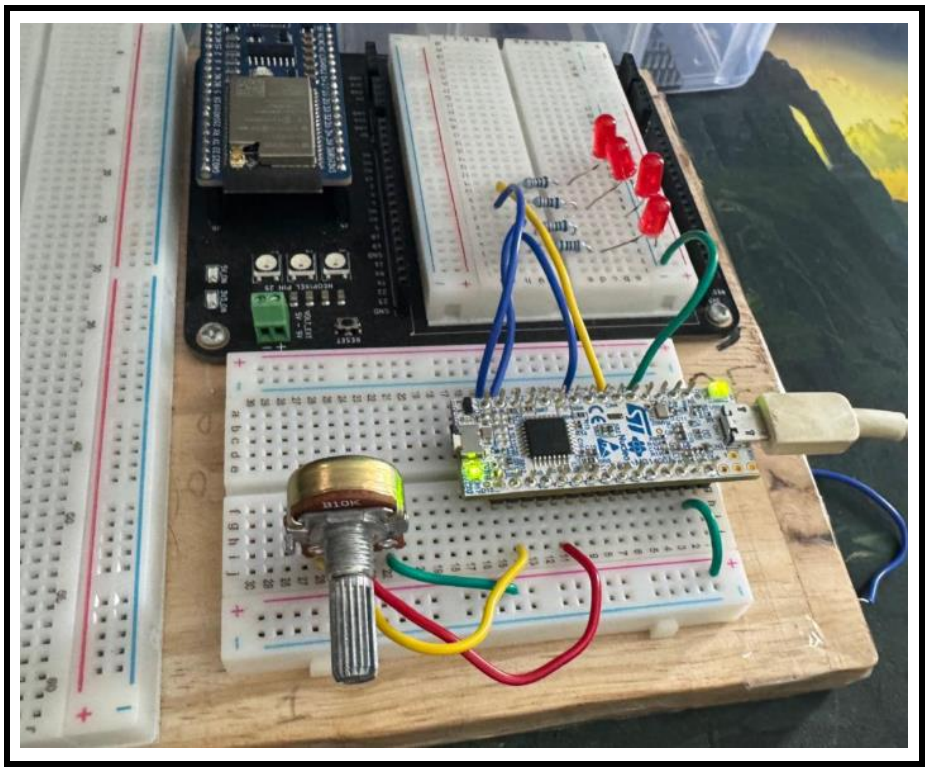


Ilustración 10- circuito armado: la tarjeta, el potenciómetro de 10 kΩ y los cuatro LEDs con sus resistencias.

Video de la práctica:

En el siguiente enlace se muestra el funcionamiento de la aplicación y la modificación del paso 5.

https://youtu.be/kMPtcK0O5WY

Resultados Teóricos:

Diseño del circuito

El esquemático del enunciado ubica los LEDs en PE8 a PE11 y el potenciómetro en PC1. Esos pines pertenecen a la tarjeta de evaluación del STM32F303, no a la Nucleo-G431KB. Esta última solo tiene 32 pines y sus conectores sacan únicamente pines de los puertos A y B, más PF0 y PF1, que están compartidos con el oscilador [2]. Tampoco trae LEDs de colores ni potenciómetro integrados, así que el circuito hubo que armarlo entero en protoboard.

Para no cambiar el armado de siempre, se reutilizaron los pines de LED del laboratorio anterior, y el potenciómetro se puso en PA1. La asignación quedó así:

Elemento	Pin	Rótulo de la tarjeta
LD1	PB0	D3
LD2	PB4	D12
LD3	PB5	D11
LD4	PB6	D6
Potenciómetro	PA1	A1

Cada LED va del pin a una resistencia de 220 Ω, luego al LED y a tierra, así que enciende con un 1 lógico. Con un LED rojo, de unos 2 V de caída, la corriente sale:

$$I = (3.3\ V - 2.0\ V) / 220\ \Omega \approx 5.9\ mA$$

El potenciómetro es de 10 kΩ y lineal, igual que el del esquemático. Sus dos extremos van a 3V3 y a tierra, y la pata del medio a PA1.

Símbolos del programa

Con todo el programa en ensamblador, no hay estructuras de CMSIS y las direcciones se definen a mano con EQU. Se tomaron del mapa de memoria y de las tablas de registros del manual de referencia [1]:

Símbolo	Valor	Qué es
RCC	0x40021000	base del módulo de reloj
AHB2ENR	0x4C	offset del registro de relojes del bus AHB2
GPIOA	0x48000000	base del puerto A, el del potenciómetro
GPIOB	0x48000400	base del puerto B, el de los LEDs
ADC2	0x50000100	base del segundo convertidor
ADC12COM	0x50000300	base de los registros comunes de ADC1 y ADC2

Los offsets de los registros del ADC también salen del manual: ISR en 0x00, CR en 0x08, CFGR en 0x0C, SMPR1 en 0x14, SQR1 en 0x30 y DR en 0x40. El registro común CCR está en 0x08 a partir de ADC12COM [1].

El canal del ADC

El anexo del enunciado pone PA1 en modo analógico y, al mismo tiempo, pide el canal 1 en SQR1. Esas dos líneas se contradicen. En el STM32G431, PA1 no es el canal 1 sino el canal 2 del ADC2; el canal 1 corresponde a PA0 [8].

Con el anexo así, el ADC acaba midiendo un canal que no es el del potenciómetro, de modo que girarlo no altera el valor leído. En el programa se corrigió a SQR1 = (2<<6), es decir, una sola conversión en el canal 2.

El orden de la configuración

El anexo deja CFGR, SQR1 y CCR configurados y recién después saca el ADC del deep power down. Ese orden no funciona: mientras el ADC está en deep power down su circuitería está apagada y lo que se escriba en sus registros se pierde, de modo que toda la configuración anterior queda sin efecto [1].

Para cerciorarse, se revisó el orden que sigue la propia librería de ST. En HAL_ADC_Init, lo primero es salir del deep power down y encender el regulador interno con su espera; el reloj común y la configuración van después [7]. El programa adoptó ese mismo orden:

Paso	Registro
1. Relojes del puerto A y del ADC12	RCC->AHB2ENR
2. PA1 como entrada analógica	GPIOA->MODER
3. Salir del deep power down	ADC2->CR, bit 29
4. Encender el regulador y esperar	ADC2->CR, bit 28
5. Reloj del convertidor (HCLK/4)	ADC12_COMMON->CCR
6. Resolución de 10 bits y OVRMOD	ADC2->CFGR
7. Una conversión en el canal 2	ADC2->SQR1
8. Tiempo de muestreo del canal 2	ADC2->SMPR1
9. Habilitar y esperar ADRDY	ADC2->CR bit 0, ADC2->ISR

Al paso 8 se le sumó una línea que el anexo no incluye. El tiempo de muestreo por omisión es demasiado corto, y con un potenciómetro de 10 kΩ la lectura tiembla. Con SMP del canal 2 en su valor más largo la lectura se asienta y los LEDs dejan de titilar en los umbrales.

Lectura y encendido de los LEDs

La subrutina adc_read lleva a cabo una conversión completa: pone ADSTART en ADC2->CR, espera la bandera EOC de ADC2->ISR y devuelve en r0 lo que haya en ADC2->DR. Como se escogió resolución de 10 bits, ese número va de 0 a 1023.

La subrutina leds_write toma ese valor y compone el patrón. Primero apaga los cuatro LEDs con la máscara TODOS, y después va comparando contra los umbrales, encendiendo un LED más en cada uno. Los umbrales salen de dividir el recorrido en quintos:

$$1024 / 5 = 204.8 \rightarrow 205, 410, 614 \text{ y } 819$$

Como las comparaciones van de menor a mayor y cada una salta a la escritura si el valor todavía no llega, el resultado es el de la tabla del enunciado:

Recorrido	Valor del ADC	LEDs encendidos
0 a 1/5	0 a 204	ninguno
1/5 a 2/5	205 a 409	LD4
2/5 a 3/5	410 a 613	LD3 y LD4
3/5 a 4/5	614 a 818	LD2, LD3 y LD4
4/5 a 1	819 a 1023	los cuatro

El patrón entero se escribe de un solo golpe al final, en la etiqueta escribir, con un único str a GPIOB->ODR. Así los cuatro LEDs cambian a la vez y no se alcanzan a ver estados intermedios.

El símbolo __ARM_use_no_argv

Con el programa ya corregido, la tarjeta seguía muerta: ni los LEDs ni ninguna prueba de parpadeo respondían, aunque el proyecto compilaba y se grababa sin errores. El problema estaba antes de main.

Antes de invocar a main, la librería de C prepara los argumentos argc y argv. Para conseguirlos usa semihosting, que es un mecanismo de depuración: el programa ejecuta la instrucción BKPT 0xAB y queda detenido esperando a que un depurador le responda. Corriendo sola, sin depurador conectado, la tarjeta se queda ahí parada y nunca llega a main.

Si el main está en C, el compilador añade por su cuenta un símbolo llamado __ARM_use_no_argv, que le avisa a la librería que esa función no recibe argumentos y que no los vaya a buscar. Con el main en ensamblador nadie lo agrega, y hay que declararlo a mano.

Esto se verificó cotejando los archivos objeto de ambas prácticas. El main.o del laboratorio anterior, escrito en C, define el símbolo en una sección propia; el de esta práctica, en ensamblador, no lo tenía, y el ejecutable resultante incluía las rutinas de semihosting y tres instrucciones BKPT que el anterior no tenía:

Laboratorio #7 (main en C)	Laboratorio #8 (main en ensamblador)
define __ARM_use_no_argv	no lo definía
sin rutinas de argumentos	incluía __ARM_get_argv y _sys_command_string
1 instrucción BKPT	3 instrucciones BKPT

La solución consistió en cuatro líneas al final del archivo, que crean la sección con el símbolo y lo exportan:

```
        AREA    |.ARM.use_no_argv|, DATA, READWRITE, ALIGN=2
        EXPORT  __ARM_use_no_argv
__ARM_use_no_argv
        DCD     1
```

Con eso el enlazador descarta las rutinas de semihosting, la librería llama a main directamente y el programa corre solo, sin depurador.

Modificación del paso 5

El paso 5 pide ignorar el dato convertido y comprobar que mover el potenciómetro ya no cambie nada. Se logra comentando una sola línea, la única que escribe al registro de datos del puerto:

```
;       str  r2, [r1,#ODR]
```

El ADC sigue convirtiendo y leds_write sigue componiendo el patrón en r2, pero ese valor nunca llega al puerto. Los cuatro LEDs se quedan apagados, que es como los dejó leds_config al arrancar, y girar el potenciómetro de extremo a extremo no produce ningún cambio.

Conclusiones:

- El código de ejemplo hay que probarlo, no solo traducirlo. El anexo del enunciado traía dos errores que solo aparecen al correrlo: pedía el canal 1 para un pin que es el canal 2, y configuraba el ADC antes de sacarlo del deep power down, donde lo que se escribe se pierde.
- Cuando algo falla, conviene dividir el problema en lugar de seguir cargando. Hacer parpadear primero el LED que ya trae la tarjeta separó de una vez tres posibles culpables: el armado, el ADC y el programa, y mostró que el que fallaba era el tercero.
- Poner el main en ensamblador acarrea una consecuencia poco evidente: la librería de C busca los argumentos de main por semihosting y, sin depurador, la tarjeta se traba antes de llegar a main. El símbolo __ARM_use_no_argv, que el compilador agrega solo cuando el main es de C, hay que declararlo a mano.

Bibliografía:

[1] STMicroelectronics, «STM32G4 series advanced Arm-based 32-bit MCUs», Reference manual RM0440, Rev. 9, marzo de 2025, capítulos 9 y 21. [En línea]. Disponible en: https://www.st.com/resource/en/reference_manual/rm0440-stm32g4-series-advanced-armbased-32bit-mcus-stmicroelectronics.pdf

[2] STMicroelectronics, «STM32G4 Nucleo-32 board (MB1430)», User manual UM2397, Rev. 2, septiembre de 2019. [En línea]. Disponible en: https://www.st.com/resource/en/user_manual/um2397-stm32g4-nucleo32-board-mb1430-stmicroelectronics.pdf

[3] STMicroelectronics, «STM32 Cortex-M4 MCUs and MPUs programming manual», Programming manual PM0214. [En línea]. Disponible en: https://www.st.com/resource/en/programming_manual/pm0214-stm32-cortexm4-mcus-and-mpus-programming-manual-stmicroelectronics.pdf

[4] J. Fajardo, «Práctica de Laboratorio: Convertidor Analógico-Digital», Microprocesadores, Universidad Galileo, octubre de 2026.

[5] Arm Limited, «armasm User Guide», Arm Compiler 6. [En línea]. Disponible en: https://developer.arm.com/documentation/

[6] Arm Limited, «Arm Compiler for Embedded User Guide: semihosting», Arm Compiler 6. [En línea]. Disponible en: https://developer.arm.com/documentation/

[7] STMicroelectronics, «stm32g4xx_hal_driver», función HAL_ADC_Init en Src/stm32g4xx_hal_adc.c. [En línea]. Disponible en: https://github.com/STMicroelectronics/stm32g4xx_hal_driver

[8] STMicroelectronics, «STM32G431x6/x8/xB datasheet», DS12589, tabla de definiciones de pines, columna de funciones adicionales. [En línea]. Disponible en: https://www.st.com/resource/en/datasheet/stm32g431kb.pdf